

Rapport : Projet-MIDL-1

- AUTEURS : LAMON Mathis, SALLEY Hugo
 - Commentaires et explications
 - ANNEXES
-

L1 : Implantation de la procédure d'élimination de la théorie DO (obligatoire)

Votre fonction prend comme entrée une formule du langage de DO, par exemple $F1 = \forall x. \forall y. \forall z. x < y \wedge x < z \rightarrow y < z$, et elle décide si la formule est vraie (c'est le cas ici) ou fausse.

— T1 prise en main des structures de données et fonctions de base (§ 5.1.1)

(Hugo) Suite a la réunion du 24 octobre je me suis alors penché sur une nouvelle structure pour les différents opérateurs, quantificateurs , etc...

« def (F:formula) → list [list [Formule]] forme normale disjontive »

([formule_to_list](#)) J'ai alors cherché une représentation avec des listes et pour moi le choix a été de mettre un string en début de liste pour identifier le type d'élément, puis plus tard je pense parcourir ces listes avec des "for each" sur des "slicing" comme dans [str_list](#). J'ai alors passé plusieurs heures à implémenter tout cela mais je me suis rendu compte que la syntaxe était encore beaucoup plus lourde. J'ai alors pour la deuxième fois tous recommencé et cette fois-ci, j'ai alors choisi de simplement changer la classe [BoolOp](#) pour qu'elle ait comme attribut un tableau. Et voilà tout a été réglé.

J'en ai aussi profité pour commencer une doc avec [Doxygen](#).

(Mathis) J'ai implémenté les fonctions [nnf](#) et [dnf](#) qui transforment une formule de la logique du premier ordre en forme normale négative pour la première et en forme normale disjonctive pour la seconde. J'ai aussi rajouté un jeu de test dans le fichier pour vérifier les différents cas (formule simple, formule complexe, formule déjà formatée,...).

— T2 implantation de la procédure de décision (§ 5.1.2)

Nous avons cherché différents algorithmes pour transformer ou tester si des formules sont des 'close' et pour les faire devenir des formes 'prénexe' juste avant de se rendre compte de cette remarque :

« Remarques : Vous pouvez supposer que la formule est déjà close et en forme prénexe. Il ne reste donc que les étapes (2) et (3). Bien sûr, les extensions décrites en § 5.2 introduisent des fonctions et constantes, mais vous pouvez les ignorer dans un premier temps. »

(Mathis) Pour implémenter la procédure de décision, nous avons d'abord avec Hugo de faire comme expliqué par dessus. Mais, en y travaillant de mon côté, je me suis essayé à faire une fonction par étape pour avoir une procédure claire et un fonctionnement facile à utiliser.

Nous nous sommes réunis le 3 Janvier afin d'implémenter les fonctions de la tâche 2. Nous nous sommes penchés sur l'annexe fournie et sur les cours de Philosophie des Sciences pour construire nos fonctions. On passe dans toute la formule pour transformer les quantificateurs universels en existentiels (Etape 0), on a mixé les étapes 1 et 2 du pré-traitement dans la fonction *push_negation* qui remplace la fonction *nnf*. Après avoir transformé la formule en DNF (étape 3), on tire les quantifications existentielles à l'intérieur de la disjonction (étape 4). La fonction *process_formula* permet le traitement récursif d'une formule pour en éliminer les quantificateurs selon les 4 fonctions décrites au-dessus.

(Hugo) J'ai par la suite de ce que nous avons fait ajouté des tests pour les fonctions *remove_forall*, *push_negation*, *to_dnf_list* et *process_formula* (notamment du pire cas).

(Mathis) Le 19 janvier et 20 janvier, après avoir demandé des précisions à Mr.Strecker, j'ai implémenté les fonctions relatives au hypothèses passés sous silences jusque là comme indiqué dans le sujet ("Remarques : Vous pouvez supposer que la formule est déjà close et en forme prénexe. Il ne reste donc que les étapes (2) et (3). Bien sûr, les extensions d'écrites en § 5.2 introduisent des fonctions et constantes, mais vous pouvez les ignorer dans un premier temps. "). J'ai commencé par les fonctions pour vérifier et transformer une formule en formule close avec des quantificateurs universels. Je suis ensuite passé sur la fonction de conversion d'une formule en forme prénexe. Cela a été compliqué car il y a des cas vraiment pas simples qu'il faut traiter, j'ai donc implémenté les cas simples comme le non et pour les conjonctions et disjonctions, je me suis aidé de gemini surtout pour les cas où une variable est liée dans une partie de la conjonction et libre dans l'autre. J'ai donc rajouté une fonction qui renomme une variable dans une formule afin d'avoir pas de problème de collision. J'ai ajouté des fonctions de tests pour la fonction de conversion et j'ai aussi remis en forme toute les fonctions pour qu'elles soient plus simple à comprendre.

La veille du rendu, je me suis penché sur le fichier de tests. J'ai remis tout dans l'ordre pour que ce soit lisible. J'ai aussi remis en forme le fichier de base car il manquait des choses par rapport à la syntaxe que Hugo avait mis en place qui pourraient causer des soucis de compréhension par la suite et j'ai aussi ajouté deux fonctions qui pourraient nous être utile par la suite, une fonction qui retourne la priorité d'un opérateur et une seconde qui met un formule sous parenthésage minimal, pour l'instant elle renvoie qu'un string mais je prévois pour plus tard de permettre de renvoyer un type Formula ce qui

peut être intéressant. J'ai ajouté donc les fonctions de tests qui vont avec ces dernières.

(Hugo) Pour peaufiner le travail, j'ai configuré *Doxygen* afin d'obtenir une documentation propre à partir des *docstrings* que nous avons déjà renseignées au fur et à mesure de l'écriture des différentes fonctions. Pour la consulter, il suffit de lancer cette commande depuis la racine du projet :

xdg-open Documentation/html/index.html

— T3 finition : tests, rapport et soutenance (§ 5.1.3)

L2 : Extension de DO à l'arithmétique des rationnels (facultative)

Tandis que la théorie DO de base admet uniquement la comparaison de deux variables (comme

$x < z$), nous nous plaçons ici dans la théorie des ordres pour l'arithmétique des nombres rationnels.

Une formule dans cette théorie est $F2 = \forall x. \exists y. \exists z. 3 * x - z < y \wedge z < 5 * x + y$.

— T4 Rapport : modifications de la procédure de base pour l'arithmétique (§ 5.2.1)

— T5 Implantation de la procédure pour l'arithmétique (§ 5.2.2)

L3 : Preuve constructive dans DO + arithmétique des rationnels (très facultative)

Avoir un résultat vrai / faux n'est parfois pas très convainquant pour un être humain. Il vous

est demandé ici de justifier la correction d'une formule. Libre cours est laissé à votre imagination,

mais voici une piste : vous pouvez créer un jeu qui est capable de répondre à tout défi de montrer

la correction d'une formule valide (est-ce que $F2$ est vrai pour $x = 2$? oui, pour $y = -1$ et $z = 8$)

et de réfuter une formule invalide.

— T6 Rapport : version constructive pour l'arithmétique (§ 5.3.2)

— T7 Implantation de la version constructive (§ 5.3.2)

ANNEXES :

(Hugo) Prise de note de la première réunion.

Séance midi 24 octobre :

def (F:formula) → list [list [Formule]] forme normale disjontive

on peut utiliser par construction par compréhension

elim de quantificateur (pour tout)

Remplacer non existe x non

nnf (non a ou b) → nnf non a et nnf non b (forme normale conjonctive)

on pourrait garder en stock le nombre de négation traverser

(on peut prendre un bool)

si impair on fait une négation sinon positif

Bibliographies :

FORMULES DE LA LOGIQUE :

https://fr.wikipedia.org/wiki/Forme_normale_n%C3%A9gative

https://fr.wikipedia.org/wiki/Forme_pr%C3%A9nexe

IA Génératives :

Copilote : Correction d'erreur dans le code , écriture/formatage de tests

Gemini : Analyses des attentes, proposition/conseil de structures